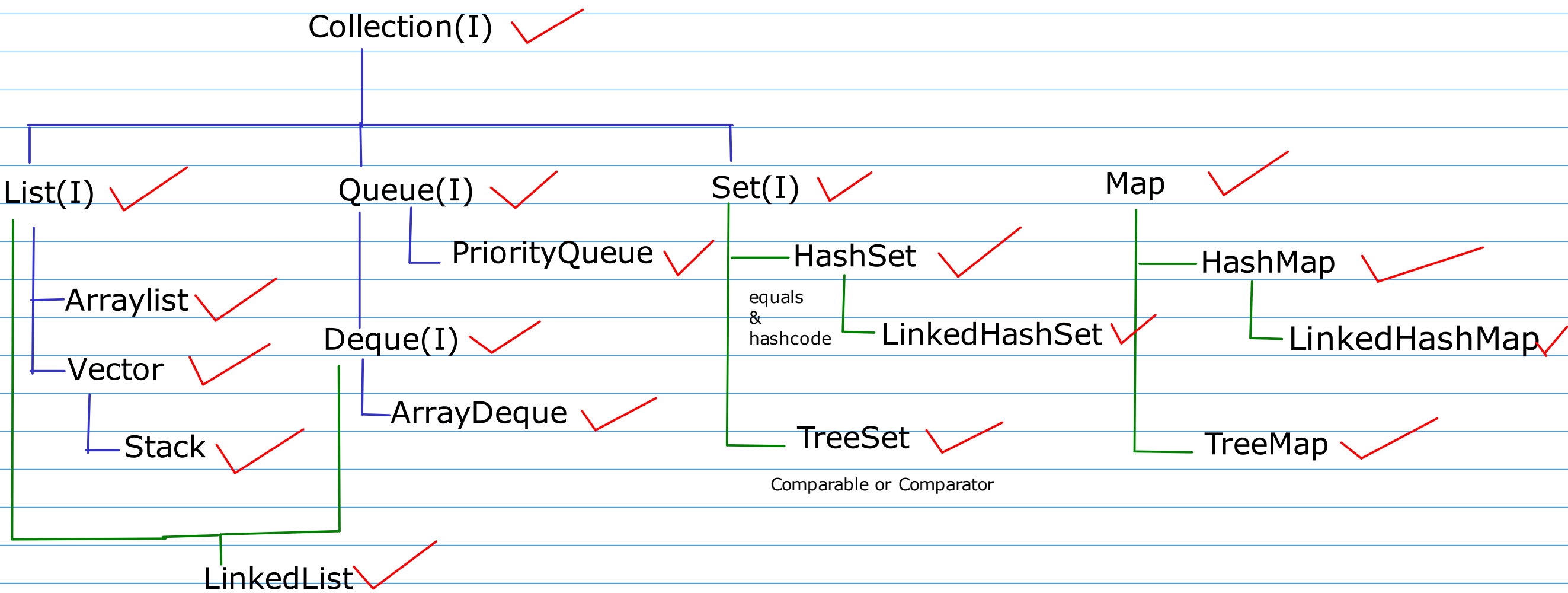




Arrays.sort()
Collections.sort()
Collections.addAll()

```
interface Collection{  
    // helper -> static  
    static void sort(){  
  
    }  
}
```

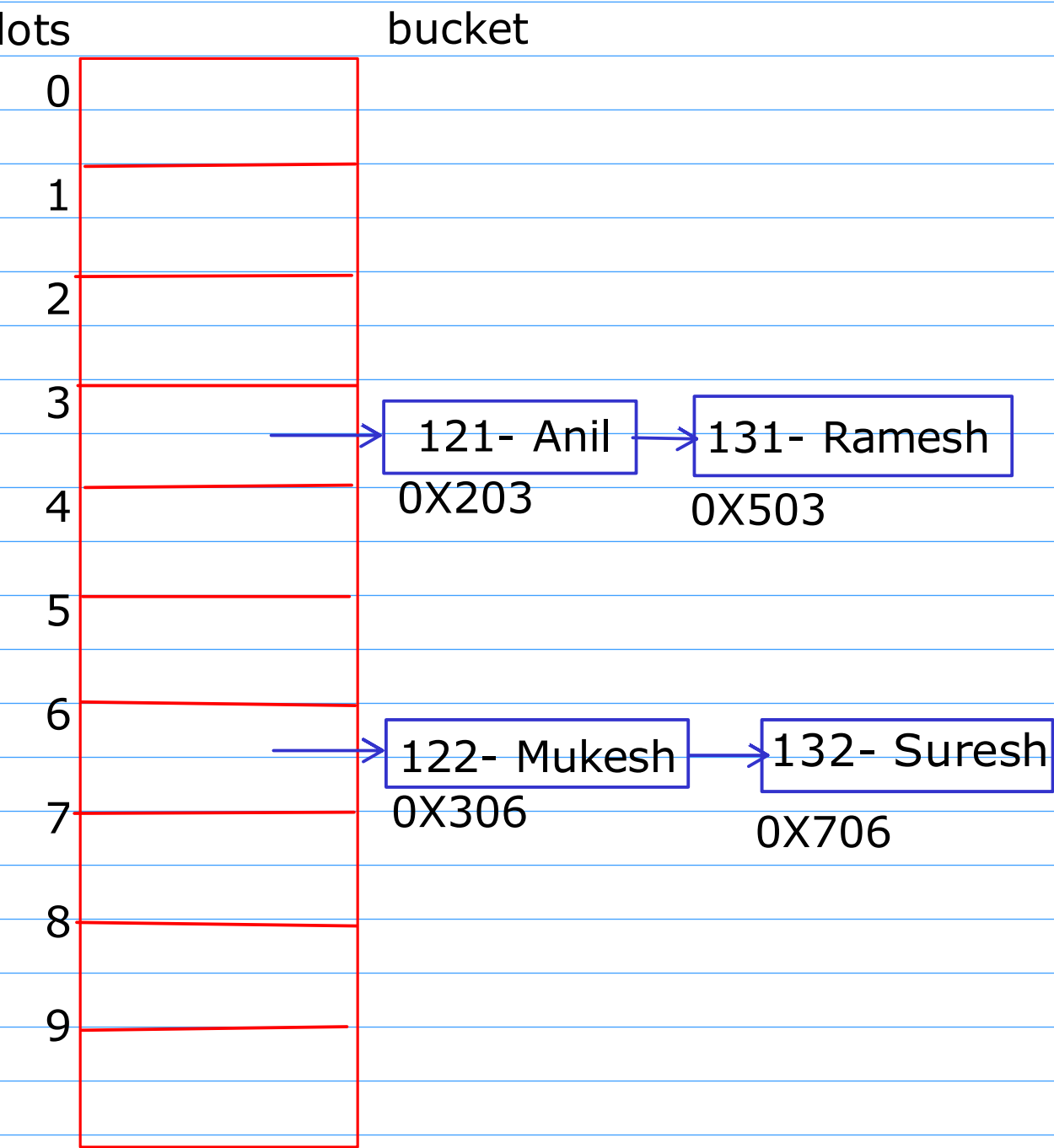
- `Predicate<T>` : test: T -> boolean
boolean test(T obj);
- `Function<T,R>` : apply: T -> R
R apply(T obj);
- `BiFunction<T,U,R>` : apply: (T,U) -> R
R apply(T o1, U o2);
- `UnaryOperator<T>` : apply: T -> T
T apply(T obj);
- `BinaryOperator<T>` : apply: (T,T) -> T
T apply(T o1, T o2)
- `Consumer<T>` : accept: T -> void
void accept(T obj);
- `Supplier<T>` : get: () -> T
T get();

Arrays.sort(arr,Comparator)
int compare(T o1,T o2)

- Method design to implement
- Implemenetation of the functional interface can be done using lambda expression

HashTable DS

```
int hashFunction(){
    return id * 3
}
```



```
slot = hashFunction()%noOfSlots
slot  = hashFunction()%10
```

$$= (131 * 3) \% 10$$
$$= 393 \% 10$$
$$= 3$$

$$= (121 * 3) \% 10$$
$$= 363 \% 10$$
$$= 3$$

$$= (122 * 3) \% 10$$
$$= 366 \% 10$$
$$= 6$$

LoadFactor = NoofElements(10) / NoOfSlots(10)

$$= 10 / 10$$
$$= 1$$

$$Lf = 5 / 10$$
$$= 0.5$$

$$Lf = 20 / 10$$
$$= 2$$

If <= 1 -> Open addressing
if lf>1 -> seperate Chaining

$$= (132 * 3) \% 10$$
$$= 396 \% 10$$
$$= 6$$

Set -> Map

- HashMap<K,V> -> HashSet<K,null>
- LinkedHasMap<K,V> -> LinkedHashSet<K,null>
- TreeMap<K,V> -> TreeSet<K,null>

Java 8 Interface